

UNIVERSIDADE FEDERAL DE LAVRAS
Departamento de Ciência da Computação
Disciplina: Inteligência Artificial

Classificação KNN na Base Iris

Implementação Hardcore vs. Scikit-learn — Análise Comparativa de Desempenho

Autores

Gabriel Soares
Lucas Rodrigues

Professor

Ahmed Ali Abdalla Esmin

Lavras, Minas Gerais
2026

1. Introdução

O algoritmo K-Nearest Neighbors (KNN) é um dos métodos de classificação supervisionada mais fundamentais em Aprendizado de Máquina, baseado no princípio de que amostras próximas no espaço de atributos tendem a pertencer à mesma classe. Sua simplicidade conceitual o torna um excelente ponto de partida para o estudo de conceitos centrais como métricas de distância, viés-variância e normalização de dados.

Este trabalho tem como objetivo implementar o KNN "do zero" (versão hardcore), sem o uso de bibliotecas de Aprendizado de Máquina, e compará-lo diretamente com a implementação de referência do scikit-learn (KNeighborsClassifier), utilizando o mesmo conjunto de dados, split e pré-processamento, a fim de validar a corretude da implementação própria e discutir as diferenças práticas entre uma solução educacional e uma solução de produção.

2. Metodologia

A base de dados utilizada foi a clássica Iris, composta por 150 amostras, 4 atributos numéricos (comprimento e largura de sépala e pétala) e 3 classes balanceadas (setosa, versicolor e virginica).

2.1 Implementação Hardcore

- cálculo da distância euclidiana entre cada par de amostras;
- busca dos k vizinhos mais próximos por ordenação das distâncias;
- votação majoritária entre os vizinhos, com critério de desempate;
- construção da matriz de confusão e cálculo das métricas (acurácia, precisão e revocação) de forma manual.

2.2 Implementação com Scikit-learn

Utilizou-se a classe KNeighborsClassifier da biblioteca scikit-learn, que internamente emprega estruturas de dados otimizadas (KD-Tree ou Ball-Tree) para acelerar a busca dos vizinhos mais próximos, eliminando a necessidade de comparar cada amostra de teste com todas as amostras de treino individualmente.

2.3 Condições de comparação justa

- Split treino/teste: 70% / 30%, estratificado por classe;
- random_state = 42, garantindo reprodutibilidade e paridade entre as duas versões;
- Normalização dos atributos via StandardScaler, aplicada de forma idêntica em ambas as implementações;
- Mesmos valores de k avaliados: 1, 3, 5 e 7.

3. Resultados

A tabela a seguir apresenta a acurácia, precisão média e revocação média para cada valor de k , lado a lado para as duas implementações.

k	Acurácia (hardcore)	Precisão média	Revocação média	Acurácia (sklearn)	Precisão média	Revocação média
1	0.9333	0.9444	0.9333	0.9333	0.9444	0.9333
3	0.9111	0.9298	0.9111	0.9111	0.9298	0.9111
5	0.9111	0.9298	0.9111	0.9111	0.9298	0.9111
7	0.9333	0.9444	0.9333	0.9333	0.9444	0.9333

Como esperado, o tempo de execução também foi comparado, refletindo a diferença de eficiência algorítmica entre uma busca por força bruta e o uso de estruturas otimizadas:

Implementação	Tempo total (4 valores de k)	Observação
Hardcore (Python puro)	≈ 0.07 s	Distância calculada ponto a ponto
Scikit-learn	≈ 0.03 s	KD-Tree / Ball-Tree otimizados

4. Análise e Discussão

- **Equivalência de resultados:** As métricas obtidas são idênticas entre as duas implementações em todos os valores de k avaliados, o que confirma a corretude da lógica implementada na versão hardcore — o algoritmo reproduz fielmente o comportamento da biblioteca de referência.
- **Padrão de erro:** O único erro sistemático observado ocorre entre as classes versicolor e virginica, nunca envolvendo a classe setosa. Esse padrão é esperado, uma vez que a setosa é linearmente separável das demais, enquanto versicolor e virginica apresentam sobreposição natural nas medidas de pétala e sépala.
- **Efeito do valor de k :** Os valores de $k=1$ e $k=7$ apresentaram acurácia levemente superior a $k=3$ e $k=5$ neste split específico. Trata-se de uma particularidade da amostra de teste sorteada (`random_state=42`), não de uma tendência geral do comportamento do KNN em função de k — outros splits poderiam produzir um ranking diferente.
- **Desempenho computacional:** A implementação hardcore consumiu aproximadamente 0.07s contra 0.03s do scikit-learn, para os quatro valores de k testados. A diferença decorre do uso de estruturas otimizadas (KD-Tree/Ball-Tree) pela biblioteca, que evitam o cálculo de distância ponto a ponto em Python puro. Com apenas 150 amostras essa diferença é irrelevante na prática, mas tende a se tornar significativa em bases de maior escala, onde a complexidade $O(n)$ por consulta da abordagem força-bruta se torna proibitiva.

5. Conclusão

As duas implementações produziram resultados equivalentes em termos de qualidade de classificação — acurácia, precisão e revocação — validando a corretude da implementação hardcore do KNN. A vantagem da biblioteca scikit-learn está concentrada na eficiência computacional, decorrente do uso de estruturas de indexação espacial, e não na qualidade da predição em si, já que o algoritmo subjacente é o mesmo.

Este exercício evidenciou, na prática, dois conceitos centrais do Aprendizado de Máquina: o efeito do hiperparâmetro k no compromisso entre viés e variância da classificação, e a importância da normalização dos atributos para que a distância euclidiana reflita corretamente a proximidade real entre as amostras no espaço de características.